

Deploying Xray-core in a Home Lab (UDM Pro + Proxmox + Kubernetes)

Introduction: What is Xray-core and Why Use It?

Xray-core is a powerful network proxy platform (a fork of V2Ray) designed to **bypass internet censorship**, act as a secure VPN/proxy, and protect privacy. It supports multiple protocols (VMess, VLESS, Trojan, Shadowsocks, etc.) and transports (TCP, WebSocket, gRPC, QUIC, etc.), allowing you to tunnel traffic in ways that are hard to detect and block. Unlike traditional VPNs, Xray's protocols (with TLS encryption) are difficult for firewalls to identify or filter ¹. Common use cases for Xray include:

- **VPN/Proxy for Privacy:** Encrypt your internet traffic and route it through your home server to protect against eavesdropping and tracking.
- **Censorship Circumvention:** Access blocked websites and apps by tunneling through restrictive networks (e.g. bypassing the Great Firewall) ².
- **Obfuscation:** Make your traffic **look like normal HTTPS** to avoid detection, using techniques like TLS with domain fronting, WebSocket camouflaged as web traffic, or the latest Xray feature *REALITY* which mimics real TLS handshakes ³.

In a home lab setup, you can run Xray-core on your own equipment (ensuring full control of data) and integrate it with your networking gear (like a UniFi UDM Pro router). Below we'll provide a step-by-step guide to deploy Xray-core in this environment and configure it for robust, secure operation.

Deployment Options for Xray-core

You have several options to deploy Xray-core in a Proxmox-based home lab: directly on a VM, inside an LXC container, or as a containerized app in Kubernetes. We'll cover each:

Option 1: Deploying Xray-core on a Proxmox VM

Running Xray on a virtual machine provides full isolation and a standard Linux environment (ideal for using systemd and the official install script). Steps:

1. **Prepare the VM:** In Proxmox VE, create a new VM (e.g. Ubuntu 22.04 or Debian 12 x64) with at least 1 CPU, 1-2 GB RAM, and 10+ GB disk. Ensure the VM is bridged to your LAN so it can receive traffic from the UDM Pro.
2. **Install system updates and basic tools:** SSH into the VM and update it:

```
sudo apt update && sudo apt upgrade -y
sudo apt install curl wget -y
```

This ensures the system is up-to-date and has `curl` / `wget` for downloading Xray.

1. **Install Xray-core using the official script:** The Xray project provides a one-step installation script that downloads the latest release, installs it to the proper directories, and sets up a systemd service. Run the following command on the VM:

```
bash -c "$(curl -L https://github.com/XTLS/Xray-install/raw/main/install-release.sh)" @ install
```

This command will download and execute the installer. It will place the Xray binary and default config in the filesystem according to FHS standards (e.g. `/usr/local/bin/xray`, config in `/usr/local/etc/xray/`, geoip and geosite data files in `/usr/local/share/xray/`, and systemd service units) ⁴ ⁵. After completion, Xray-core is installed as a service but not yet running.

Alternative: If you prefer manual installation, you can download the **Xray-core release** from GitHub and install it. For example:

```
VERSION="Xray-linux-64.zip"
wget https://github.com/XTLS/Xray-core/releases/latest/download/$VERSION
unzip $VERSION -d xray-install
sudo install -m 755 xray-install/xray /usr/local/bin/xray
sudo install -d /usr/local/etc/xray && sudo cp xray-install/example/config.json /usr/local/etc/xray/
```

and so on. However, the script above automates these steps and sets up systemd for you.

1. **Obtain a domain name and TLS certificate:** To securely expose Xray to the internet, you should use TLS. Acquire a domain or subdomain that points to your home's public IP (you can use dynamic DNS if your IP changes). Then obtain a TLS certificate for that domain, for example using **Let's Encrypt**. One convenient method is using the ACME script (`acme.sh`):

2. Install `acme.sh` and issue a cert for your domain (using HTTP challenge or DNS challenge). For instance:

```
acme.sh --issue -d vpn.yourdomain.com --webroot /var/www/html
```

3. Once issued, install the cert to a path Xray can access. In the Xray docs example, they copy the cert to `~/xray_cert/`:

```
mkdir -p ~/xray_cert
acme.sh --install-cert -d vpn.yourdomain.com --ecc \
  --fullchain-file ~/xray_cert/xray.crt \
  --key-file ~/xray_cert/xray.key
chmod +r ~/xray_cert/xray.key
```

This yields `xray.crt` and `xray.key` (the private key is made world-readable here for simplicity) ⁶ ⁷. You could also use Certbot; the key point is to have `certificateFile` and `keyFile` ready for Xray.

4. **Configure Xray-core (server config):** Edit Xray's config file (default at `/usr/local/etc/xray/config.json`). At minimum, define an **inbound** for your chosen protocol (e.g. VLESS or VMess) and an **outbound** to `freedom` (direct to internet). Below is an example of a **VLESS + WebSocket + TLS** server config (with placeholders):

```
{
  "inbounds": [{
    "port": 443,
    "protocol": "vless",
    "settings": {
      "clients": [
        { "id": "<YOUR-UUID>", "flow": "xtls-rprx-vision", "level": 0 }
      ],
      "decryption": "none",
      "fallbacks": [
        { "dest": 80 } // Fallback to local port 80 (e.g. a web server) for
unknown traffic
      ]
    },
    "streamSettings": {
      "network": "ws",
      "security": "tls",
      "tlsSettings": {
        "certificates": [{
          "certificateFile": "/home/youruser/xray_cert/xray.crt",
          "keyFile": "/home/youruser/xray_cert/xray.key"
        }]
      }
    },
    "wsSettings": {
      "path": "/websocket" // WebSocket path for camouflaged traffic
    }
  }
],
}
```

```
"outbounds": [{ "protocol": "freedom" }]
}
```

In this config:

- **Protocol:** VLESS (a lightweight stateless proxy protocol). We enable XTLS with `flow: xtls-rprx-vision` for efficiency ⁸ ⁹.
- **Port 443 with TLS:** Xray will handle TLS using the provided certificate files ¹⁰. It's listening on TCP+WebSocket (`network: "ws"`) which mimics normal WebSocket traffic.
- **WebSocket path:** set to `"/websocket"` (you can choose a random path; this must be known to clients). This traffic looks like WebSocket over HTTPS (difficult to distinguish from regular web traffic).
- **Fallbacks:** The `"fallbacks": [{ "dest": 80 }]` means any unrelated traffic on port 443 is forwarded to port 80 (e.g., you could run a simple web server on port 80 to serve a dummy site) ¹¹. This way, if someone accesses your domain in a browser, they hit a benign website, but Xray clients using the correct WebSocket path get proxy service. This **hides the proxy** in plain sight.

Tip: Use a unique UUID for each client. Generate one with `xray uuid` or online. Keep the config JSON well-formed; Xray logs will help debug any syntax errors.

1. **Start and enable the Xray service:** Use systemd to manage Xray. Start Xray and check status:

```
sudo systemctl start xray
sudo systemctl status xray
```

Ensure it shows **active (running)** with no errors ¹². Enable it to auto-start on boot:

```
sudo systemctl enable xray
```

(The install script already created the service unit at `/etc/systemd/system/xray.service` ¹³.) You can use `systemctl restart xray` on config changes and `journalctl -u xray --follow` to tail logs.

1. **Test locally:** From another machine or container on the network, try to connect using an Xray client (e.g., `v2rayN` on Windows or `v2rayNG` on Android). Use your home lab's domain (e.g. `vpn.yourdomain.com`), port 443, the matching protocol (VLESS in this example), the UUID, and the WebSocket path and TLS on. If configured correctly, you should be able to browse the internet through your home Xray server. Check Xray's logs (`/var/log/xray/access.log` if enabled, or stdout via `journalctl`) for incoming connections and any errors.

Option 2: Deploying Xray-core in a Proxmox LXC Container

Using an LXC container is more lightweight than a full VM, but comes with a few considerations:

1. **Create the LXC container:** In Proxmox, create an LXC using a Debian or Alpine template. Allocate sufficient resources (1 CPU, 512MB-1GB RAM). **Important:** Xray's installer expects systemd by default, which doesn't run in an unprivileged LXC without special handling. Easiest solutions:
2. Create the LXC **privileged** and enable nesting (in Proxmox CT Options, set `Nestling: 1`). This allows systemd to run inside the container.
3. Or, use an Alpine Linux container and follow Xray's OpenRC installation (the Xray-install repo provides OpenRC scripts for Alpine/Gentoo) ¹⁴. Alpine's init is different, but Xray can be set up to run under it.
4. **Install Xray in the LXC:** If you went with a Debian/Ubuntu LXC and enabled nesting, you can use the same **official script** method as above (`bash -c "$(curl -L ...)" @ install`) ¹⁵. This will place Xray in `/usr/local` and attempt to set up systemd service. Since this is a container, ensure the container has `systemd` running (you might need to boot the container with systemd as init - privileged LXC makes this easier). After running the script, verify that `systemctl status xray` works inside the container.

If using Alpine (no systemd), there is a separate installation path:

- Use the OpenRC install script from the Xray-install repo (see **README_alpinelinux** in the repo). It will install Xray and set up an OpenRC service. For example (in Alpine):

```
apk add curl
curl -L https://github.com/XTLS/Xray-install/raw/main/install-release.sh -
o install.sh
sh install.sh -u root
```

(Adjust for OpenRC usage as per the repo instructions.)

- **Configure Xray:** Inside the LXC, edit the Xray config (`/usr/local/etc/xray/config.json`). This will be essentially the same as for a VM. You can even bind Xray to the container's IP on port 443. If the container is on a bridge with the UDM, it will have a LAN IP (e.g., 192.168.1.100). We will forward ports to it in the UDM section.
- **Start Xray:** For Debian LXC with systemd, use the same `systemctl start xray` and enable on boot. For Alpine, start the service with OpenRC (`rc-service xray start` and add to default runlevel). Ensure Xray is running in the container.
- **Networking for LXC:** Proxmox containers share the host kernel. By default, an LXC will use the host's firewall rules. Ensure Proxmox's host firewall isn't blocking the ports (or simply disable firewall

on the container network interface in Proxmox settings for that CT). Usually, if the UDM forwards traffic to the container's IP, it should reach the service.

Using LXC can save resources, but if you run into issues (especially with systemd), a VM might be simpler. Either way, once Xray is running in the container, the remaining steps (domain, certificate, etc.) are the same.

Option 3: Deploying Xray-core as a Kubernetes Workload (Optional)

If your home lab uses Kubernetes for orchestration (perhaps with Proxmox VMs as worker nodes), you can run Xray-core in the cluster. This is advanced and optional. Two approaches are common:

- **Using a Docker Container:** Xray has official container images (e.g., `ghcr.io/xtls/xray-core`) and community images (`teddysun/xray`). You can use these in a Deployment or StatefulSet. The container will run the Xray binary; you provide a config via a volume mount.
- **Helm or YAML manifests:** As of now (2025/2026), there isn't an official Helm chart for Xray-core, but you can deploy it with a custom manifest. For example, create a **ConfigMap** for `config.json` and a **Deployment** that mounts it:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: xray-config
data:
  config.json: |
    { ... (your Xray JSON config here, similar to above) ... }

---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: xray-server
spec:
  replicas: 1
  selector:
    matchLabels: { app: xray-server }
  template:
    metadata:
      labels: { app: xray-server }
    spec:
      containers:
        - name: xray
          image: ghcr.io/xtls/xray-core:latest
          args: ["-c", "/etc/xray/config.json"]
          ports:
```

```

- containerPort: 443
  name: https
volumeMounts:
- name: config
  mountPath: /etc/xray
volumes:
- name: config
  configMap:
    name: xray-config

```

This will run Xray in a pod, listening on port 443. We pass the config via a ConfigMap (mounted at `/etc/xray/config.json`). The `ghcr.io/xtls/xray-core` image runs as non-root by default and supports multiple CPU architectures ¹⁶. Ensure the container has `cap_net_bind_service` or runs as root if you want it to bind 443 (the official image might be non-root; if so, you can either use port 8443 in-container and map it, or use `securityContext` to allow binding low ports).

- **Expose Xray in K8s:** In a home cluster, you likely don't have a cloud LoadBalancer. You have a few choices:
- **NodePort:** e.g., expose 443 as a NodePort (like 30443). Then configure UDM to forward WAN 443 to the node's 30443. (This introduces an uncommon port internally, which is fine.)
- **HostPort or HostNetwork:** You can let the Xray pod bind the host's port 443 directly. For example, set `hostNetwork: true` and container port 443 (ensure no other service on host uses 443). This way, the node itself listens on 443 for Xray traffic. UDM can forward to the node's IP on 443.
- **Ingress Controller:** You could also use a Kubernetes Ingress with TLS passthrough or terminating TLS at the ingress. This is complex (many ingress controllers don't easily allow raw TCP/TLS passthrough for custom protocols). If you already have an Nginx ingress, you might configure a TCP stream for Xray. But most likely, NodePort or HostPort is simpler.
- **Kubernetes Liveness/Readiness Probes:** To ensure high availability, you can add a liveness probe to restart the pod if Xray crashes. For instance:

```

livenessProbe:
  tcpSocket:
    port: 443
  initialDelaySeconds: 30
  periodSeconds: 60

```

This simply checks if the TCP port is open. Xray doesn't provide an HTTP health endpoint by default, so a TCP check is the simplest probe (if the process hangs, the port likely won't accept). Also consider `restartPolicy: Always` (default) and perhaps a startup probe if Xray needs time to initialize.

After deploying, test from an external client as usual. Running Xray in Kubernetes is likely overkill for a single-instance proxy (and introduces complexity in exposing it through the UDM), but it can be useful if you already containerize everything or want to manage via Helm/Flux/etc.

Note: If you have a multi-node K8s cluster, ensure the Xray pod either runs on a specific node (use `nodeSelector` or `DaemonSet`) or that all nodes can accept the forwarded traffic (if using MetalLB for a service LoadBalancer). For a home lab, often a single-node K3s or MicroK8s setup is used, which simplifies this.

Integrating Xray with UniFi UDM Pro (Networking & Firewall)

Your UDM Pro is the gateway router for your home. We need to **forward external traffic** to the Xray server and do it securely:

1. Port Forwarding (NAT): On the UDM Pro's UniFi Network Controller interface, set up a **port forwarding rule** to direct incoming Xray traffic to your server. For example, if Xray is listening on `TCP 443` on internal IP `192.168.1.100`:

- **Source:** Any (or restrict to specific source IP ranges if you know clients' IPs – typically not, since you may roam).
- **Port:** 443 (WAN/external)
- **Destination IP:** 192.168.1.100 (the VM or LXC running Xray)
- **Destination Port:** 443 (internal)

In UniFi's UI: go to **Settings > Routing & Firewall > Port Forwarding** and create a new rule. Enter the port and IP details as above (enable "Match" on TCP, and likely you leave UDP blank unless you use Xray's UDP features separately). The UDM will then NAT any inbound TCP 443 to your Xray server ¹⁷.

If you run Xray on a different port or multiple ports (some setups use 80, 443, 8443, etc.), repeat for those. But generally, using port 443 is recommended for stealth (common port).

2. Firewall Rules: The UDM Pro automatically creates corresponding firewall allow rules for port forwards by default ("Firewall/NAT" handling in UniFi ensures the forwarded port is open). However, double-check in **Firewall > WAN In** rules that there isn't a rule blocking your forwarded port. If you use UniFi Threat Management/IDS/IPS, monitor it – IDS may flag unusual traffic. Since Xray traffic looks like normal TLS, it usually isn't flagged, but high-volume or non-web TLS might occasionally show up as "Misc" traffic. If the IDS gives false alarms, you might need to create an exception or disable certain signatures.

Optionally, for extra security, you could add a firewall rule to only allow specific source IPs to reach port 443 on the UDM. This is only practical if you yourself have a static IP when traveling, etc., which is rare. Generally you'll keep it open to all, trusting the Xray's authentication.

3. DNS Considerations: Ensure your domain (e.g. `vpn.yourdomain.com`) is pointing to your home's public IP. If your ISP IP is dynamic, use a dynamic DNS service and update the DNS regularly. The domain's A/AAAA record should be correct so that clients can reach your UDM's IP via the domain. (The TLS certificate we set up is tied to that domain.)

4. TLS/Encryption Setup: We already obtained a Let's Encrypt cert and configured Xray with it. This means Xray will perform the TLS handshake with clients. The UDM is just forwarding the encrypted traffic – it does not inspect or terminate TLS. This is good for privacy (UDM sees only encrypted data on 443) and for

camouflaging the traffic. Make sure the UDM isn't doing any SSL/TLS interception (it typically doesn't for port forwards). In effect, the UDM treats the Xray traffic like any HTTPS connection to an internal server.

5. Using a Reverse Proxy (Alternative): If for some reason you prefer not to run TLS in Xray itself, you could place a reverse proxy (like Nginx or Caddy) on the VM to handle TLS and forward to Xray on a localhost port. For example, Nginx can listen on 443 with your cert and forward WebSocket traffic to Xray on 127.0.0.1:12345. This is an advanced setup and usually not necessary, since Xray can handle TLS just fine. One benefit of using something like **Caddy** is automatic certificate renewal via Let's Encrypt, but you can achieve the same with a cron job for acme.sh or Certbot on the VM.

Most home lab users keep it simple: UDM forward -> Xray handles TLS. This works well.

6. Testing from outside: Try connecting from an external network (e.g., mobile data or a friend's internet) with your Xray client. If it connects and you can browse, the UDM's forwarding is working. If not, recheck that the port forward rule is enabled and correct. You can also use online port-check tools on port 443 to see if it's open. Note: some ISPs block inbound 443 (rare for residential, more common for mobile hotspots). If 443 is blocked, you might try alternate ports (like 8443 or 4433) but keep in mind those are less "common" and might stick out. Ideally, use 443 or 443 + other well-known ports (many run additional fallbacks on 80 or 53, but 80 will conflict if you host a site, and 53 UDP is DNS which could be suspicious).

7. UDM Pro TLS Hostname Issues: One thing to mention – UDM (and most routers) don't do SNI routing. If you also host a *different* service on 443 on the same WAN IP (like a webserver for another domain), you can't forward port 443 to two different internal servers based on hostname without a reverse proxy. In such cases, you'd need to consolidate behind one proxy or use different ports. A common approach is to let Xray have 443 and use the fallback feature to route innocuous traffic to a webserver (as we did in config). That way Xray on 443 can "share" the port with a web service by inspecting the TLS SNI or HTTP path. Xray's built-in fallback can send unknown requests to a local web service (perhaps running on the same VM, port 80 or another port) ¹⁸. This effectively serves a website and Xray on the same port. Ensure the domain you use for Xray either is solely for Xray or any web service on it is meant as cover.

In summary, once UDM Pro is forwarding the port and Xray is handling TLS, the integration is done. All the heavy lifting (encryption, access control, routing to internet) is done by Xray on the VM/container. The UDM's role is just to pipe traffic through while enforcing basic network security at the perimeter.

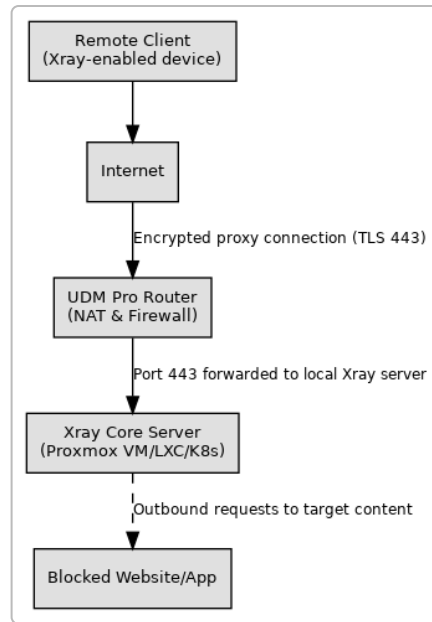


Diagram: Network flow in home lab – External client connects to your domain (TLS 443) -> UDM Pro forwards to Xray server -> Xray decrypts and proxies the traffic out to blocked sites. Legitimate TLS traffic is indistinguishable from the outside.

Example Xray Configurations (Protocols and Modes)

Xray-core supports various protocols and obfuscation techniques. Here are a few common example configurations and their typical use:

- **VLESS + XTLS + WebSocket:** This combines a **VLESS** inbound (auth by UUID, no static encryption overhead) with **XTLS** (Xray's optimized TLS, "Vision" flow) and transports data over **WebSocket** (often masked as normal HTTPS traffic). This is what we configured above. It's currently a popular choice for performance and stealth. The config we showed is an example of VLESS+WS+TLS (with XTLS) for the server, and clients would use matching settings (WebSocket transport, TLS, specifying the path and SNI). This setup makes the proxy traffic look like a user visiting a web site (especially if you run it on 443 with a valid cert). XTLS Vision further reduces TLS overhead for proxy connections, improving speed ⁸.
- **Trojan (TCP + TLS):** Trojan protocol is essentially a stealth proxy that imitates a regular TLS connection. The client connects with a password (like a Trojan "shared secret") and from the outside it looks like HTTPS traffic to a normal server. Xray supports Trojan as an inbound/outbound. A simple Trojan server config in Xray might look like:

```
{
  "inbounds": [{
    "port": 443,
    "protocol": "trojan",
    "settings": {
```

```

    "clients": [{ "password": "your-strong-password" }]
  },
  "streamSettings": {
    "network": "tcp",
    "security": "tls",
    "tlsSettings": { "certificates": [ { /* your cert */ } ] }
  }
}

```

Trojan only works over TLS (no UDP). Its advantage is **simplicity and camouflage** – since it uses a valid TLS handshake, some say “no distinguishable protocol fingerprint” (it just looks like a client speaking TLS to a server) ¹⁹. The server expects the correct password after TLS; if not, the connection is closed (or could be passed to a real web service, acting like an ordinary TLS server). This way, active probes are resisted because unless the probe knows the password, it can’t complete the protocol ¹⁹ ²⁰. Use Trojan if you prefer password auth over UUID, or need to use certain clients that support it. Remember to **choose a long random password** and keep TLS strong.

- **VMess (TCP or WS + TLS):** VMess is the original V2Ray protocol using UUID auth and encryption. It’s still widely used. A typical VMess over TCP+TLS server config is similar to VLESS, but with `"protocol": "vmess"` and under settings, `"clients": [{ "id": "<UUID>", "alterId": 0 }]`. VMess has an `alterId` (now often 0 with new clients, as it’s deprecated) and the data is encrypted with a dynamic key per session. Many deployments use VMess over WebSocket+TLS as well, which looks like:

```

"streamSettings": {
  "network": "ws",
  "security": "tls",
  "tlsSettings": { /* certs */ },
  "wsSettings": { "path": "/ray" }
}

```

The difference from VLESS is just the protocol name and that VMess has encryption of payload by default. VMess is considered moderately secure and still effective, but it adds a bit more overhead than VLESS. If you have legacy clients or need compatibility, you might run a VMess service. VMess also benefits from TLS to blend in. (Non-TLS VMess is not recommended on hostile networks; it can be identified.)

- **Others:** Xray also supports **Shadowsocks**, **Socks/HTTP** proxies, **gRPC** transports, **QUIC** and even integration with WireGuard (Xray can bridge WireGuard through its server) ²¹. For example, you could deploy a **VLESS over gRPC** which appears as HTTP/2 traffic, or use the new **Reality** (which is essentially VLESS+XTLS but without needing a certificate – it uses a preset public key of some domain to fake TLS, very useful if you *can’t* get a cert or domain). Covering all combinations is beyond scope, but the Xray-examples repository contains JSON samples for virtually every scenario ²² – from “VMess+WS+TLS” to “Trojan-gRPC” to “VLESS+Reality over XHTTP/3”. You can refer to those examples and adapt to your needs.

To highlight one new feature: **REALITY** – this is an Xray-specific enhancement where the server presents a fake TLS handshake imitating a real website (e.g., it can mimic *www.microsoft.com* using its public key) without needing an actual cert for that domain. The handshake looks legitimate to censors, but inside a static ECDH key is used to derive encryption. REALITY is considered state-of-the-art in 2025 for censorship circumvention ³. It's configured under `security: "reality"` with parameters like `serverNames` and `privateKey`. If you're interested, the Xray docs have a tutorial on REALITY ²¹. For a home lab, if you already have your own domain and cert, using regular TLS is fine – but REALITY is an option if you cannot expose port 80 for Let's Encrypt or want an extra layer of plausible deniability.

Multi-User & Multi-Protocol: You can run multiple inbounds in Xray simultaneously. For instance, one inbound could be VLESS on 443, another could be Trojan on 443 (since Trojan and VLESS can both run on TLS – you would differentiate by SNI or ALPN). Xray's routing can also direct traffic by SNI to different inbounds. However, for simplicity, most home setups just run one primary service (or one port with fallbacks). If you anticipate many users, you can add multiple UUIDs (clients) in the inbound settings or use a UI panel to manage users.

Config Verification: After editing configs, always verify JSON syntax (you can use `jq . /usr/local/etc/xray/config.json` to validate). Xray will refuse to start if JSON is invalid. Also note that changes to config require a service restart (`systemctl restart xray`). Monitor the log output (`/var/log/xray/error.log` if configured, or via `journalctl`) for any errors on startup or client connection attempts.

Automating Installation and Updates (Xray-install Script & Tools)

Keeping Xray up-to-date and managing it can be automated:

- **Official Xray-install script:** We used this for initial install. You can run the same script to upgrade Xray-core in the future. For example:

```
bash -c "$(curl -L https://github.com/XTLS/Xray-install/raw/main/install-release.sh)" @ install
```

again will fetch the newest release and replace the binary (it preserves your config and doesn't overwrite it). The script can also update the GeoIP/GeoSite data (`@ install-geodata`) or remove Xray. It even supports installing specific versions or beta releases (`--beta` flag) ²³ ²⁴. This is very handy – subscribe to Xray-core's GitHub releases or Telegram for updates, and run the script when a new version is out.

- **Xray binary via package managers:** If you used Homebrew on Linux (`brew install xray`) or an Alpine package, you could update via those mechanisms. But the script is the recommended method on standard Linux.
- **Automation tools:** There are third-party scripts/projects like **Xray-ONE** or **v2ray-agent** that automate interactive setup (firewall rules, config generation, cron updates). Use these with caution and only if they are well-maintained. In 2026, the official script and manual editing often suffice, but

for beginners a project like `Xray-install` or `Xray-manager` can simplify initial setup (they might prompt for domain, generate UUID, configure TLS with Let's Encrypt automatically, etc.).

- **Container updates:** If running in Docker/K8s, you can update by pulling the new image (`docker pull ghcr.io/xtls/xray-core:latest`) and recreating the container. Consider using something like Watchtower for automatic Docker image updates. In Kubernetes, update the image tag in your manifest or Helm chart and roll the deployment.
- **Service monitoring:** Ensure Xray starts on boot (`enable` was done). If you want to monitor it, you can use tools like **Monit** or systemd's restart on failure (the service unit installed includes `Restart=on-failure` by default). In a more complex setup (like with Kubernetes or a process supervisor), you'd rely on liveness probes or a watchdog process. For example, *CompassVPN* (an open source project around Xray) uses Monit to restart Xray or Nginx if they go down ²⁵. For most, systemd's inherent reliability (and the fact Xray is stable) means it'll run continuously without issues.
- **Logging:** By default, Xray's config might not log to file (the installer created log files at `/var/log/xray/access.log` and `error.log`, but they remain empty unless you add a `"log"` section in config) ²⁶. You can enable logging like:

```
"log": {  
  "access": "/var/log/xray/access.log",  
  "error": "/var/log/xray/error.log",  
  "loglevel": "warning"  
},
```

This logs client connection info (access) and warnings/errors to those files. Use `loglevel: "info"` for more verbosity (caution: can be a lot). Logging is helpful to see if connections are succeeding and if there are any errors (e.g., TLS handshake issues, wrong UUID attempts, etc.). Since these logs can grow, consider installing **logrotate** or use the script's `--logrotate` option to set up rotation ²⁷.

- **Xray-checker:** There is a community tool `xray-checker` ²⁸ which can check if a new version of Xray is available and even update it. Not strictly necessary, but if you want an automated check on a cron that's an option.

In short, treat Xray like any important service: keep it updated for the latest protocol improvements (the Xray team frequently enhances anti-censorship techniques), and monitor its status. The official script and systemd make management quite straightforward.

Security Best Practices

Finally, some security considerations for running Xray in your home lab:

- **Strong Authentication:** Use strong, unpredictable IDs or passwords for your proxy clients. For VMess/VLESS, that means using the generated UUIDs (don't use trivial ones). For Trojan, use a long random password. This prevents unauthorized use of your proxy. Xray's protocols don't have

“usernames” per se (though you can tag email in clients for identification), so the secrecy of those credentials is key.

- **Firewall Restrictions (Server Side):** Even though UDM is filtering WAN access, you can also run a software firewall (like `ufw` or raw `iptables`) on the VM/container. For instance, you might allow inbound to port 443 only and drop any other unexpected connections. If you know you will only connect from certain regions or IPs, you could even limit source IP ranges (but this can be impractical if you travel or have roaming clients). Generally, at least ensure no other high-risk services on the VM are open to WAN. Since the VM is behind NAT, it's not directly exposed except for Xray's port. Still, good hygiene: update the OS, close unnecessary ports.
- **Fail2Ban Intrusion Prevention:** **Fail2Ban** can monitor Xray logs and ban IPs that show malicious signs. For example, if someone is trying random UUIDs or passwords to use your proxy (effectively an authentication brute-force or a scanner), Xray's error log would show failures. Fail2Ban can tail those logs and add firewall rules to ban that IP for a period. Some Xray admin panels integrate Fail2Ban for this reason ²⁹. You'd need to create a custom filter for Xray logs (matching “failed handshakes” or similar messages) and a jail to ban. While not absolutely critical (a well-configured Xray will just reject invalid clients without many side effects), banning persistent offenders can reduce noise. It also helps against port scanners – e.g., if some IP hits your 443 and doesn't do a proper TLS (perhaps probing), Fail2Ban could ban after a few “bad TLS” log entries.
- **Obfuscation and Camouflage:** A key tenet of Xray is **hiding traffic patterns**. Our use of TLS+WebSocket already does this: it looks like someone downloading data from a web server. To improve camouflage:
- Use **valid TLS certificates and SNI** that match a real domain. (We did this with Let's Encrypt cert for your domain – it's real and trusted.) Tools that reset TLS or use self-signed certs can trigger suspicion; a real cert for a normal-looking domain is ideal.
- Serve or imitate a real website on that domain (even if just a landing page). With Xray's fallback or with an Nginx proxy, you can host a simple blog or a status page. That way, if the domain is accessed via browser, it returns normal content. This prevents someone from easily deducing “this domain is just a proxy”. Even a basic HTML page is fine.
- Leverage **uniform TLS fingerprints**: Xray (and its clients) support *uTLS*, meaning they can mimic the TLS fingerprint of common browsers (Chrome, Firefox, etc.) ³⁰. This is important because some censors look at the TLS ClientHello fingerprint. Ensure your client is set to use, say, Chrome's fingerprint (many Xray clients have an option for this, like v2rayNG's “fingerprint” setting). On the server, Xray by default is passive in this regard – just be sure not to disable or break it by forcing a specific cipher. Using XTLS or REALITY automatically takes care of mimicry.
- Consider **domain fronting or CDN**: This is advanced, but some configurations route Xray traffic through Cloudflare or similar CDNs (e.g., running WebSocket on a Cloudflare-proxied domain). This can hide the true server IP. If you do this, be aware of Cloudflare's terms and the performance hit (also Cloudflare might cut you off if they detect non-HTTP traffic). But some users have had success with “Cloudflare -> Nginx -> Xray” setups for added cover ³¹.

- **Traffic Shaping Evasion:** Enable **padding or split traffic** if your scenario needs it. For instance, Xray can split HTTP requests (there's a SplitHTTP transport) to evade size-based detection. These are edge cases – by default, WebSocket/TLS should be fine.
- **Resource Hardening:** Enable **BBR congestion control** on your Linux VM (to improve throughput over high-latency links). This is a kernel setting; many guides cover enabling BBR (`sysctl` config). The Xray docs mention it as an optimization ³² – it's not security per se, but helps performance which can indirectly keep your link usage under the radar (by efficiently using available bandwidth, reducing obvious throttling patterns).
- **Monitoring Usage:** Keep an eye on bandwidth usage. If you are in a bandwidth-constrained environment (some ISPs have caps or your UDM's IPS can only handle so much encrypted traffic), monitor how much data flows through your proxy. The UDM Pro has traffic stats per client. The Xray access log also can be parsed to see usage per user if you have multiple. This isn't directly a security tip, but it ensures your proxy doesn't overwhelm your network or get you into trouble with ISP for unexpected usage patterns.
- **IP Blacklisting:** Use GeoIP rules if needed. Xray can do routing based on IPs (e.g., using `geoip:cn` to block or direct traffic). If you never expect connections from certain countries to your proxy, you could drop them at the Xray level. But be cautious – many people use VPNs themselves, so the source IP might not reflect their true locale.
- **Updating and Patching:** Keep Xray-core updated (we covered this) – not only for features but security fixes. Also update the system (openssh, libraries, etc.) as usual. The smaller your VM's attack surface (only Xray port open), the lower the risk – but it's good practice to maintain the system.
- **Home Network Safety:** Since this is in a home lab, ensure the Xray server (VM or container) is on a DMZ or isolated VLAN if possible. You wouldn't want a compromised Xray instance to have unfettered access to your LAN devices. The UDM Pro allows creating VLANs; you could put the VM in an "untrusted" VLAN that only has internet access. At minimum, do not reuse the Xray VM for other critical services that, if exposed, could be compromised via the proxy.

By implementing the above, your Xray deployment should be **secure, stealthy, and resilient**. For example, CompassVPN (a project that packages Xray) uses several of these practices: Fail2Ban to block repeat offenders, Monit to auto-restart, Cloudflare integration for hiding IP, and auto-updates ³³ ²⁹ .

Conclusion

You now have a comprehensive guide to deploying Xray-core in a home lab environment with UniFi UDM Pro and Proxmox. We covered installing on a VM or container, exposing it safely through your router with TLS, and configuring various powerful proxy protocols that Xray offers. With this setup, you can enjoy censorship-free internet and secure connections from anywhere. Xray's flexibility (VMess, VLESS, Trojan, etc.) means you can tweak it to your needs – whether prioritizing speed (XTLS Vision), stealth (camouflage as web traffic), or simplicity (single-port Trojan). Always test your configuration in a controlled manner and incrementally improve security (by analyzing logs and adjusting rules).

By following this guide and referencing official examples and documentation ²², you've essentially built your own VPN/proxy server that *"penetrates everything"*. Enjoy your privacy and freedom online, backed by the power of Xray-core!

Sources and Further Reading:

- Official Project X (Xray-core) documentation and examples ² ²²
- Xray-core GitHub repository and Xray-examples configs ⁹ ³⁴
- XTLS/Xray-install script and usage guide ¹⁵ ⁵
- Community tutorials on VLESS+XTLS and REALITY usage ³ ²¹
- Security advice from CompassVPN (open-source Xray-based VPN) ²⁹ ³³

¹ ² ²² GitHub - wpdevelopment11/xray-tutorial: Set up an Xray-core VLESS proxy to access blocked websites and apps

<https://github.com/wpdevelopment11/xray-tutorial>

³ Network Censorship Circumvention – Wispy Docs

<https://wispydocs.pages.dev/network-censorship-circumvention/>

⁴ ⁵ ¹³ ¹⁴ ¹⁵ ²³ ²⁴ ²⁶ ²⁷ GitHub - XTLS/Xray-install: Easiest way to install & upgrade Xray

<https://github.com/XTLS/Xray-install>

⁶ ⁷ ⁸ ⁹ ¹⁰ ¹¹ ¹² ¹⁸ ³² [Chapter 7] Xray Server Guide | Project X

<https://xtls.github.io/en/document/level-0/ch07-xray-server.html>

¹⁶ Download and Install - Project X

<https://xtls.github.io/en/document/install.html>

¹⁷ UDM-PRO port forwarding - Ubiquiti Community

<https://community.ui.com/questions/UDM-PRO-port-forwarding/63f695bf-c2b2-49c1-b5d0-6594b13b5024>

¹⁹ ²⁰ Trojan - sing-box

<https://sing-box.sagernet.org/manual/proxy-protocol/trojan/>

²¹ ²⁸ GitHub - XTLS/Xray-core: Xray, Penetrates Everything. Also the best v2ray-core. Where the magic happens. An open platform for various uses.

<https://github.com/XTLS/Xray-core>

²⁵ ²⁹ ³¹ ³³ FAQ – CompassVPN

<https://www.compassvpn.org/faq/>

³⁰ ³⁴ VLESS WSS NGINX · Issue #167 · XTLS/Xray-examples · GitHub

<https://github.com/XTLS/Xray-examples/issues/167>